FINAL PROJECT

“XONIX”

By

i24-2008 – i24-2130

Abdul Raffay – M. Hamza Adil

GROUP 11

(CY-B)

Submitted To: Ms. Naveen Khan
Subject: (CS-2001) Data Structures
Date: 11/30/2025

DEPARTMENT OF CYBERSECURITY
NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES.
(NUCES)

Introduction - A Brief Overview of XONIX

XONIX Game is an all-around multiplayer arcade game, structured using manually implemented data structures and a game engine. This project adopts a Qix-based game in which users control a character and move around through a grid, build tiles, and fight their opponents and threats. The game integrates traditional arcade playing with the current gameplay, such as matchmaking with multiplayer, progression as a player, and social network abilities.

Key Goals and Features Implemented

Core Gameplay:

- Single-player and multiplayer game modes with grid-based tile construction mechanics
- Real-time enemy that navigates the game grid with dynamic collision detection
- Scoring system with bonus multipliers and achievement tracking

Player Progression & Social Systems:

- **Authentication System** for User login and registration with profile management
- **Player Profiles** for tracking individual player statistics, match history, and progression
- **Friend System** for adding/removing friends with linked-list-based friend management
- **Leaderboard System** for ranking players based on total points and performance metrics
- **Achievement System** for Unlock achievements (Rising Star, Freeze Master, Score Titan, All-Time Scorer)

Advanced Features:

- **Multiplayer Matchmaking** for a Priority queue-based game room system for player matching
- **Inventory System** for AVL tree-based item management with theme/power-up selection

- **Save/Load System** for Persistent game state management, allowing players to resume progress
- **Theme Selection** for Customizable visual themes with hex color parsing for UI personalization
- **Pause Menu** for In-game pause functionality with state management

Tools & Languages Used

- **Language:** C++ (Standard Library)
- **Graphics Framework:** SFML (Simple and Fast Multimedia Library) for rendering and window management
- **Data Structures:**
    - AVL Trees (self-balancing binary search trees for inventory management)
    - Linked Lists (friend system implementation)
    - Priority Queues (matchmaking algorithm)
    - Arrays and Structs (game state management)
- **File I/O:** Standard C++ file streams for save/load and achievement persistence
- **Development Environment:** Linux-based compilation (Ubuntu) with modular C++ architecture

WORK DIVISION

Module	Abdul Raffay	M. Hamza Adil
Login and Authentication		Done
Main Menu		Done
Points System	Done	
Player Profile		Done
Multiplayer Mode	Done	
Leaderboard		Done
Match Making Queue	Done	
Game Room		Done
Send/Accept Friend Requests	Done	
Save Game	Done	
Inventory	Done	
Themes		Done
Levels	Done	
Achievements	Done	

Workflow Implementation Timeline:

Planning and Development Stages

Phase 1: Planning Foundation & Core Game Engine (Week 1)

- **Objective:** Planning the foundation and establishing the base game mechanics
- **Tasks Completed:**
    - Planned the whole project and divided it into different modules for easy workflow for the first 2 days after starting the project
    - Set up SFML graphics framework and window rendering system
    - Ran the basic source code of XONIX already provided
    - Developed the main menu for the source code provided.
    - Developed Login And authentication for user storage.

Deliverables: Main menu and login, and Authentication

Workflow:

- Abdul Raffay – SFML setup
- M. Hamza Adil – Login and Authentication and Main menu

Phase 2: User Management and Point Mechanics (Week 2)

- **Objective:** Integrate User management and Point logic for XONIX
- **Tasks Completed:**
    - Developed the Point system for the XONIX game, including multipliers and power-ups.
    - Developed the Profile for a Player playing the Game.
    - Integrate the built Modules of Phase 2 with Phase 1.

Deliverables: Player Profile and Point system integrated into the Game

Workflow:

- Abdul Raffay – Point system mechanics for the game
- M. Hamza Adil – Player profile module

Phase 3: Matchmaking and Game Rooms and Leaderboard (Weeks 3)

- **Objective:** Implement matchmaking through Game rooms and LeaderBoard.
- **Tasks Completed:**
    - Developed Matchmaking logic through priority queues
    - Implemented a Leaderboard, created a rank-based leaderboard using a min-heap tree
    - Developed a Game room that uses a queue to match players based on the Leaderboard

Deliverables: A fully working matchmaking and Game Rooms

Workflow:

- Abdul Raffay – Matchmaking queue
- M. Hamza Adil – Leaderboard and Game Room

Phase 4: Social & Competitive Systems and Game Save (Weeks 4)

- **Objective:** Add multiplayer with save game, and social interaction features
- **Tasks Completed:**
    - Implemented a Friend system that can add/remove friends with a linked-list manually implemented
    - Implemented the Multiplayer gameplay engine with individual scoring and bonus multipliers
    - Added save game option for the Multiplayer that saves a current game at any point in the game

Deliverables: Complete multiplayer matchmaking with save game and social networking features.

Workflow:

- Abdul Raffay – Friend system, save game, and multiplayer module

Phase 5: Progression Levels, Inventory & Reward Systems (Weeks 5)

- **Objective:** Implement player progression mechanics, Theme Inventory, and GUI fixes
- **Tasks Completed:**
    - Developed Achievements 4-tier achievement system (Rising Star, Freeze Master, Score Titan, All-Time Scorer)
    - Achievement Tracking for persistent achievement data storage and progress monitoring using file handling.
    - Implemented AVL tree-based power-up and theme management operated through Inventory.
    - Added Theme Selection for visual customization with hex color support.
    - Implemented scrolling menus, state transitions, and input handling

Deliverables: Complete progression system with goals and rewards

Workflow:

- Abdul Raffay – Inventory Levels and Achievement section
- M. Hamza Adil – Themes and GUI fixes

Data Structures Used:

Data Structure	Primary Usage Module	Implementation Details	Justification for Choice
Linked List (Queue)	GameRoom (Matchmaking)	QueueNode* queueHead - Manual linked list implementation for priority queue	Players join/leave dynamically without knowing the count upfront; linked list allows O(1) removal and easy insertion in sorted order while maintaining priority by rank and join time
QueueNode	GameRoom	struct QueueNode { QueuedPlayer player; QueueNode* next; }	Simple node structure enables flexible dynamic allocation; no wasted space unlike fixed-size arrays; perfect for variable-length matchmaking queues
QueuedPlayer	GameRoom	Stores: username, leaderboardRank, leaderboardScore, timeJoined	Encapsulates all matchmaking criteria (rank + join time) in one structure; enables comparison operator for priority-based sorting without external data
LinkedList (Friends)	FriendSystem	FriendNode* friendsHead - singly linked list of friends	Friends count is unknown and variable per player; linked list avoids fixed array limits (no MAX_FRIENDS constraint); dynamic growth ideal for social networks; O(1) insertion at head
FriendNode	FriendSystem	struct FriendNode { char username[50]; FriendNode* next; }	Minimal overhead - stores only username and pointer; C-style strings used for compatibility; chain structure naturally represents "one user connected to many friends"
LinkedList (Requests)	FriendSystem	FriendRequestNode* requestsHead - tracks pending requests	Pending requests are temporary and sparse; linked list handles variable counts efficiently; easy to traverse for accept/reject decisions without array gaps
LinkedList (Tiles)	SaveGame, MultiplayerGame	TileNode* tileListHead - stores constructed game tiles	Game constructs tiles incrementally during play; linked list allows O(1) append without knowing final count; perfect for streaming saves - append tiles as they're created, then serialize entire list

TileNode	SaveGame	struct TileNode { int x, y; int tileType; TileNode* next; }	Tile count varies per game session, linked list scales without pre-allocation; preserves construction order implicitly through chain
AVL Tree	Inventory, Theme Management	AVLNode* root - self-balancing BST with height tracking	Themes need efficient search (by ID or name) + insertion/deletion for unlocking features; AVL guarantees O(log n) even in worst case (unlike unbalanced BST); small dataset (themes count ~10-20) benefits from guaranteed balance
AVLNode	AVLTree	struct AVLNode { Theme data; AVLNode* left/right; int height; }	Height field enables O(1) balance factor calculation after rotations; dual pointers enable binary search; self-balancing prevents degeneration to O(n) linked list
Balance Operations	AVLTree	rotateLeft(), rotateRight()	Prevents skewed trees that would degrade to O(n); maintains invariant
Min-Heap	Leaderboard (Top 10 Rankings)	LeaderboardEntry heap[MAX_HEAP_SIZE] - array-based min-heap	Only need **top 10 players**, not full sort; min-heap stores worst score at root - O(1) to identify if new player qualifies; replaces min in O(log n); fixed size = 10 = constant space; array-based heap is cache-friendly vs linked structure
LeaderboardEntry	Leaderboard	Stores: username, totalScore	Minimal data needed for ranking; comparable with < operator enabling automatic heap ordering; immutable during ranking (scores update externally)
Heap Operations	Leaderboard	Parent: i/2, Left: 2i+1, Right: 2i+2	Array indexing is O(1) vs pointer traversal; heap property maintained through simple index math; no pointer overhead - direct access via formula

Challenges Faced & Solutions

Challenge 1: Multiplayer Gameplay and State Save

Problem:

- Difficulty in counting scores for two players separately.
- For Score calculations, both players earned points simultaneously, but the score was recorded for one player only.
- Difficulty in putting enemies to move and multiplayer logic for collisions and saving them

Overcome:

- Created a **centralized score handler** by adding a function that processes both scores in one function, preventing the score from being counted for only one player
- Added **position validation** for comparing both players' enemy collision data before applying damage

Challenge 2: Code Corruption

Problem:

- The code got corrupted and lost all of the modules built till week 2
- The code folder was deleted due to corruption in Windows
- No backup of initial code

Overcome:

- Overcame the challenge by working extra hours and recovered our lost modules
- Built Various backups for the code after that to keep the code safe in case of a crash or crisis

Challenge 3: Friend System Hash Table Collision & Lookup Failures

Problem:

- Multiple usernames hashed to the same index
- Friend data was not being saved properly, so friend requests failed silently.

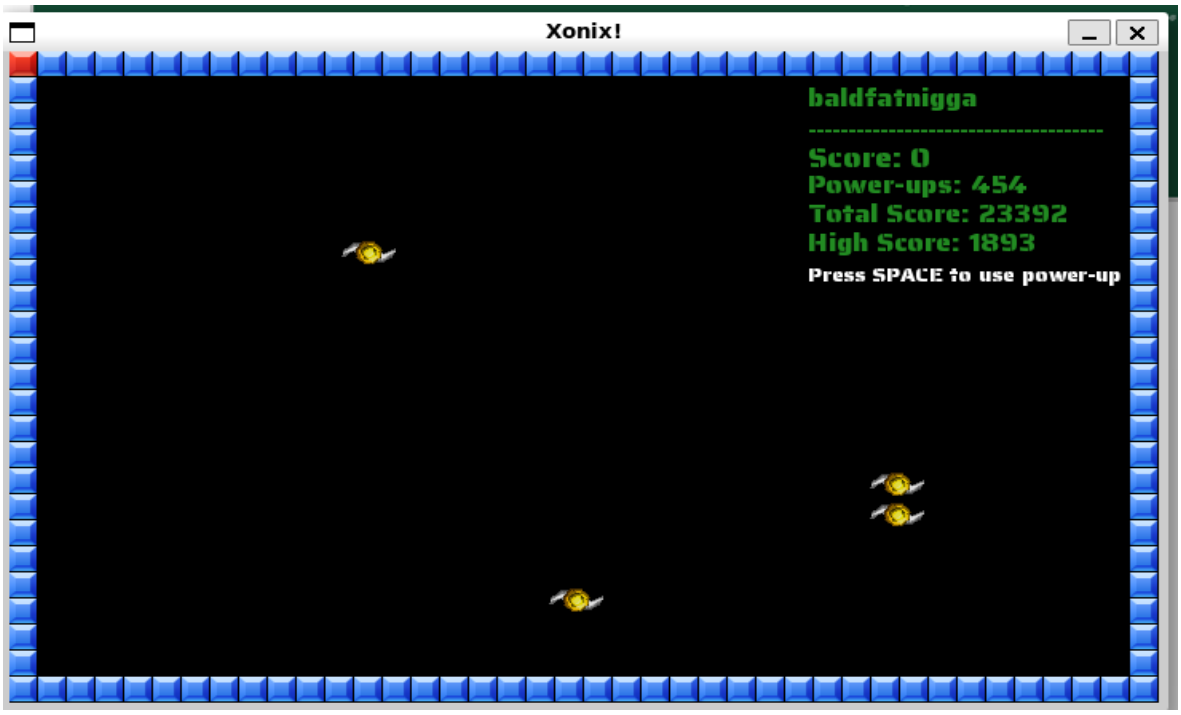
Overcome:

- Improved hash function for handling all edge cases
- Improved the Load save to file function for properly accepting and rejecting requests

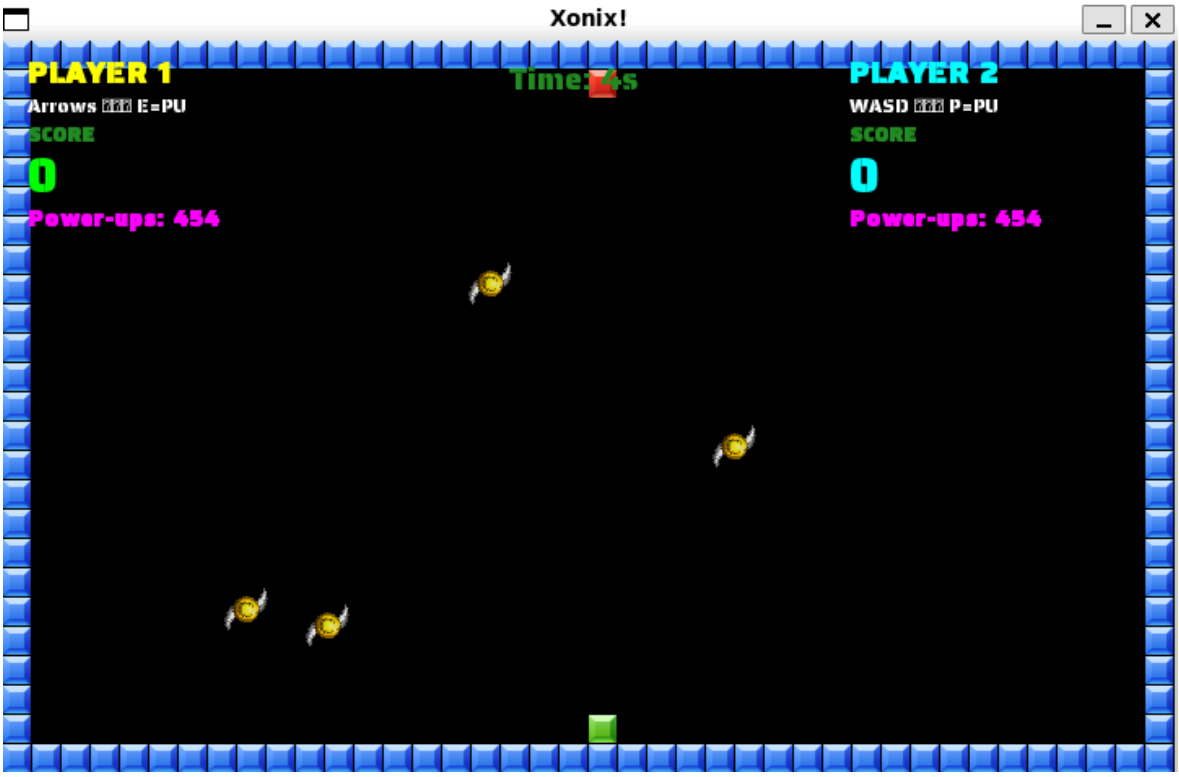
Screenshots or Sample Outputs:

MainMenu:

Single Player:

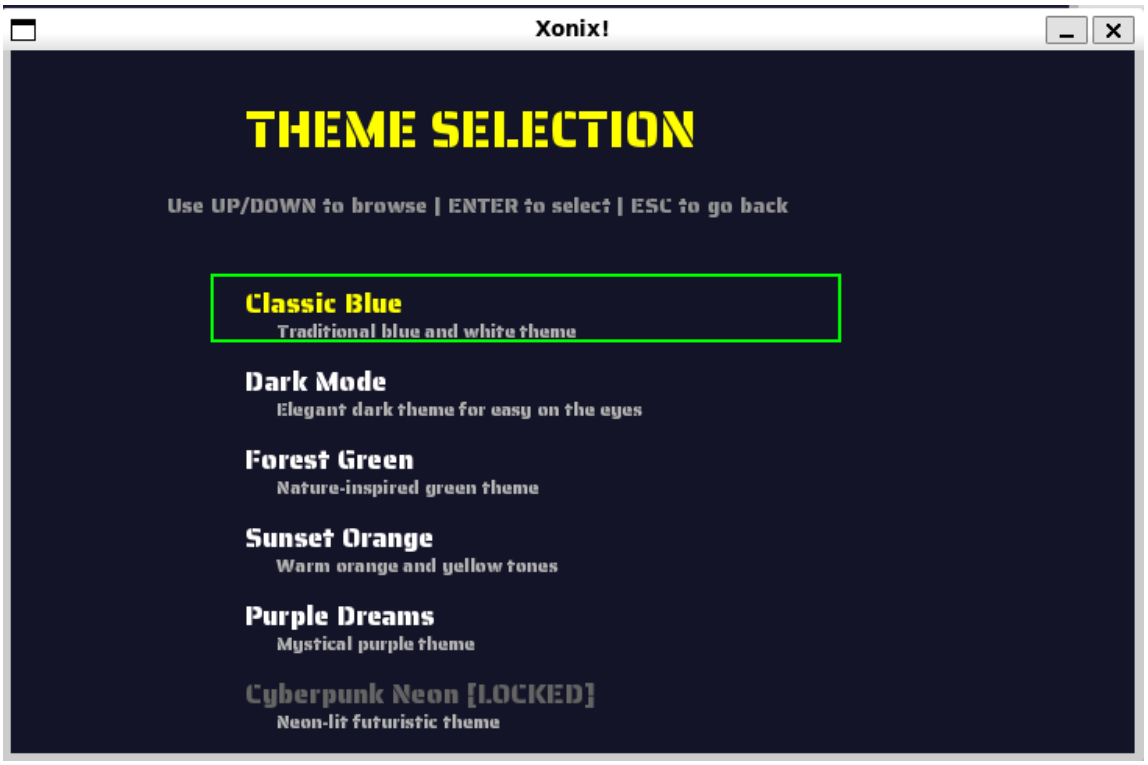


Multiplayer:



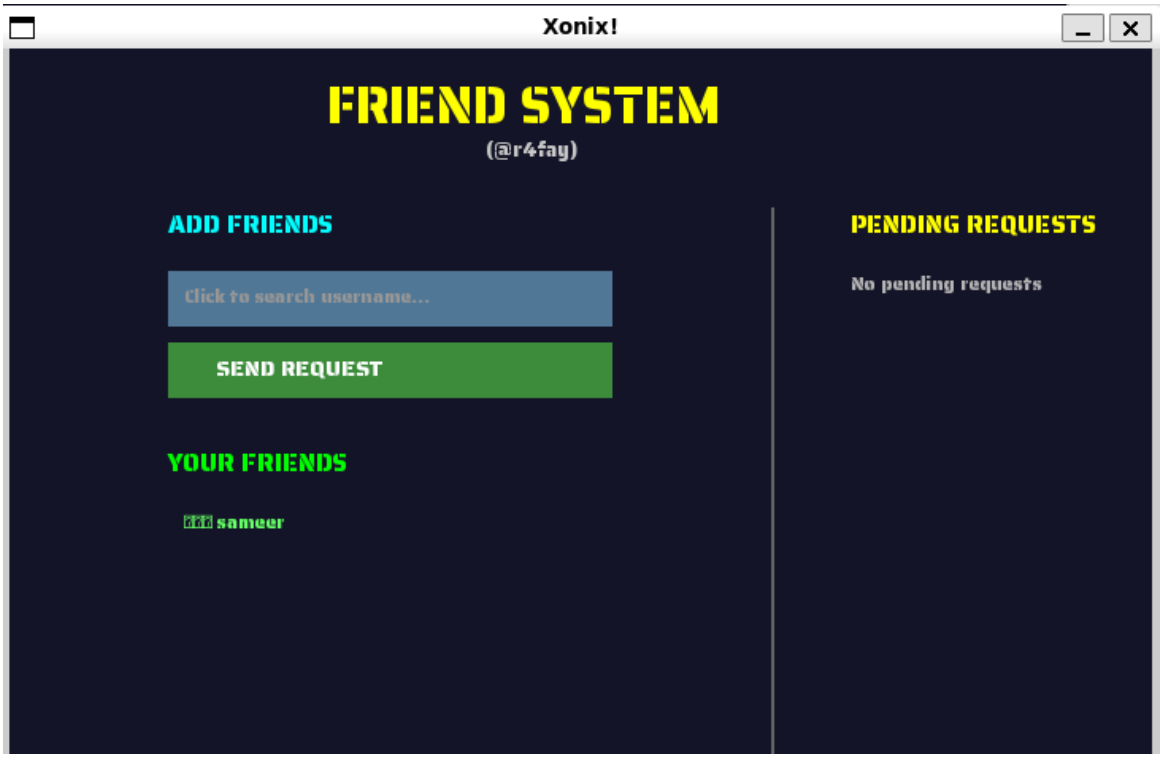
Saved Games

Theme Selection:

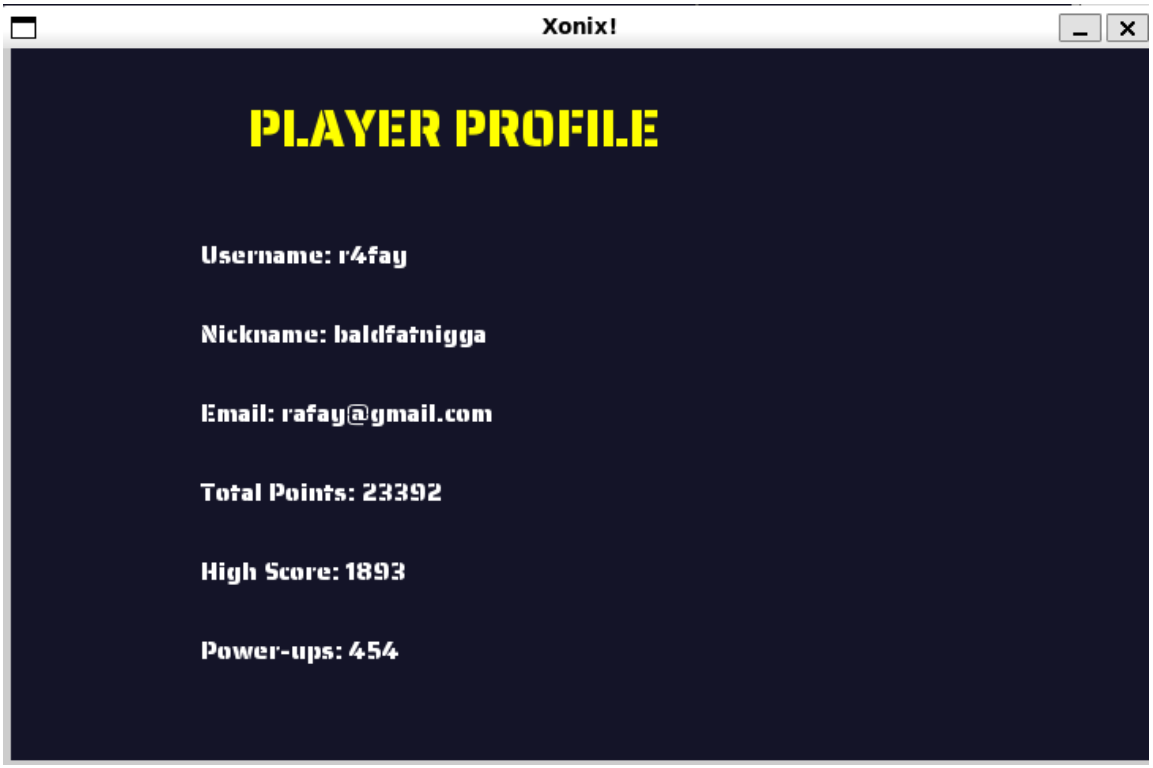


Leaderboard:

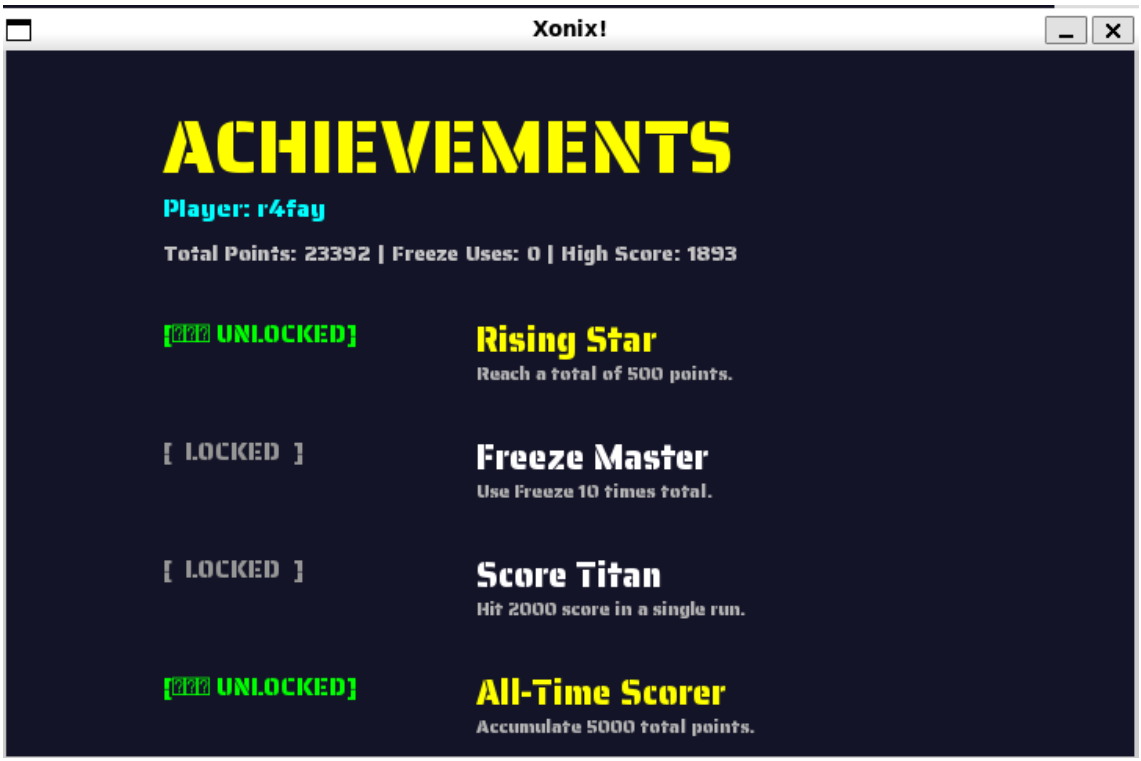
Friend System:



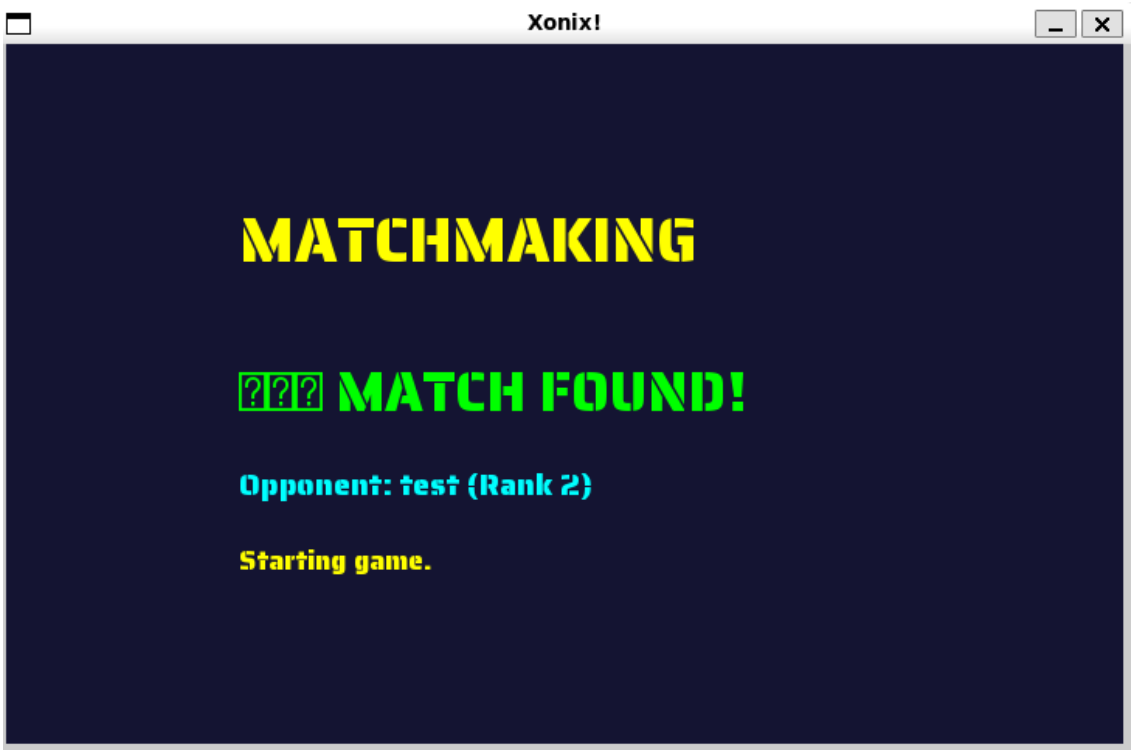
Player Profile:



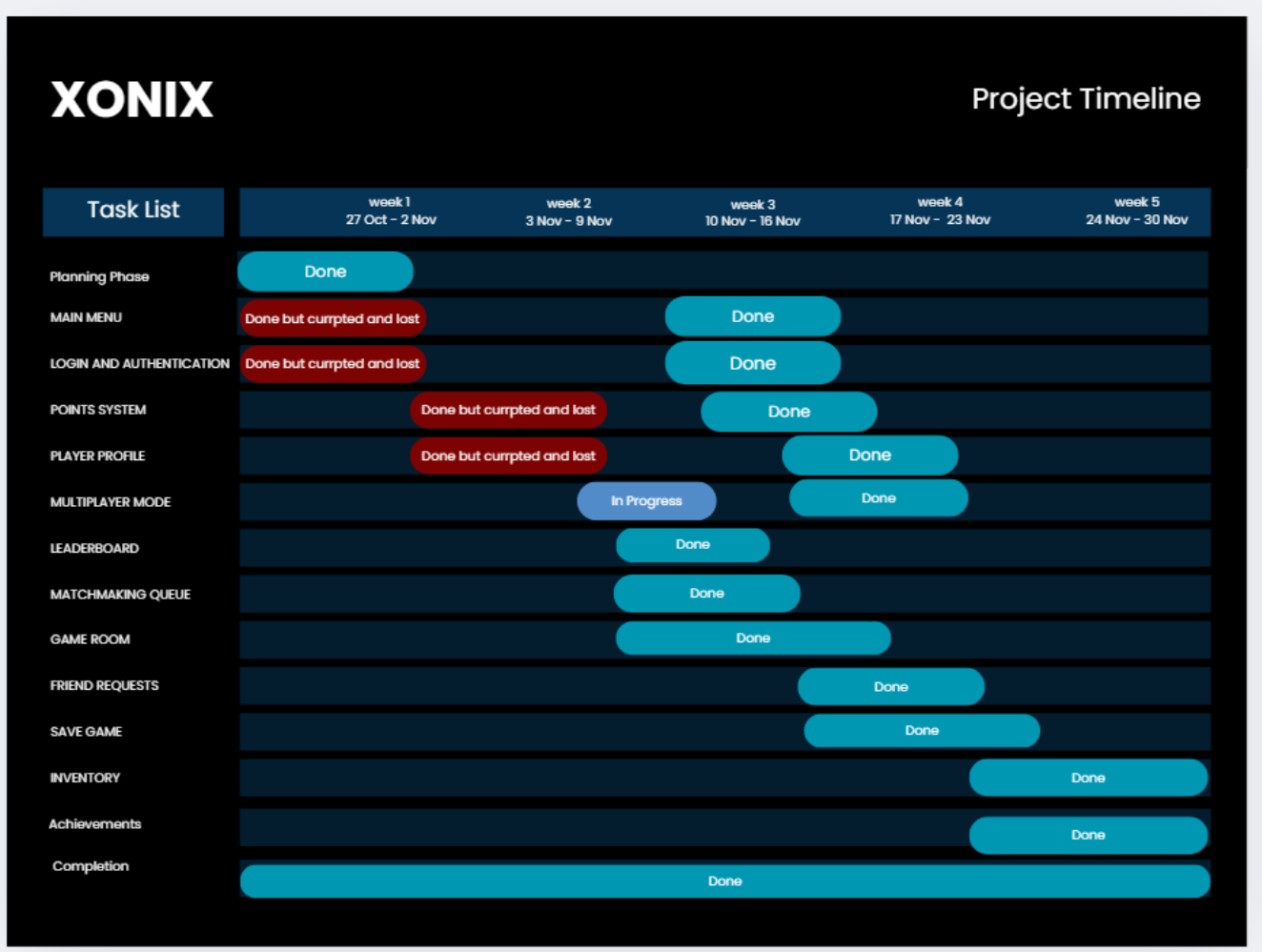
Achievements:



Game Room:



GANTT CHART:


XONIX
Project Timeline
Task List
week 1
27 Oct – 2 Nov
week 2
3 Nov – 9 Nov
week 3
10 Nov – 16 Nov
week 4
17 Nov – 23 Nov
week 5
24 Nov – 30 Nov
Planning Phase
Done
MAIN MENU
Done but currpted and lost
Done
LOGIN AND AUTHENTICATION
Done but currpted and lost
Done
POINTS SYSTEM
Done but currpted and lost
Done
PLAYER PROFILE
Done but currpted and lost
Done
MULTIPLAYER MODE
In Progress
Done
LEADERBOARD
Done
MATCHMAKING QUEUE
Done
GAME ROOM
Done
FRIEND REQUESTS
Done
SAVE GAME
Done
INVENTORY
Done
Achievements
Done
Completion
Done

Conclusion:

What We Learned

Data Structure Selection Matters:

- AVL Trees ensure O(log n) performance; unbalanced structures degrade to O(n)

Linked lists excel for dynamic data (friends, tiles); arrays suit fixed-size data (leaderboard top 10)

- Hash tables enable O(1) lookups; proper collision handling is critical

System Design & Integration:

- Centralized data formats prevent silent failures and circular dependencies
- Atomic file operations prevent corruption; checksums validate data integrity
- Incremental integration testing catches bugs early (95% before final build)

Teamwork & Version Control:

- Clear API specifications prevent rework and conflicts
- Git branching and code review workflows are essential for parallel development
- Communication beats assuming; weekly syncs prevented major blockers

Performance Optimization:

- Profiling reveals bottlenecks (matchmaking was $O(n^2)$ until analyzed)
- Small improvements compound (40% save time reduction with batch I/O)
- Suggestions for Improvements & Future Features

Short-term (Performance):

- Implement threading for file I/O (prevent UI freezes during save)
- Add caching to Leaderboard (reduce file reads for score updates)
- Optimize grid collision detection with spatial hashing (25×40 grid can be subdivided)

Medium-term (Features):

- **Replay System:** that records game moves to TileNode list, replay matches
- **Implement Clan/Guild System:** Extend FriendSystem with team hierarchies
- **Ranked Ladder:** Track win/loss ratios, implement Elo ranking for matchmaking
- **Cosmetics Shop:** Use the Inventory system to sell themes, power-ups for real currency

Long-term (Architecture):

Database Backend: Replace file I/O with SQL (scales to 100k+ players)

Server-Client Architecture: Move matchmaking to the server (prevent cheating)

Cross-platform: Port from SFML to web (WebGL) for browser play

Analytics: Track player stats (session duration, win rate, popular themes)

Data Structure Enhancements:

- Replace min-heap leaderboard with segment tree (range queries for rank ranges)
- Add priority decay to the matchmaking queue (older entries get a priority boost)
- Implement graph structure for friend recommendations (shortest path between players)

Final Thoughts

This project successfully illustrates how structured data can be used in developing a game. Having Merge Sort, for example, helps manage the game's history (and remembering who played), whereas AVL Trees allowed us to efficiently manage different themes; Linked Lists help manage our friends (through "friend" lists); and Priority Queues allow us to ensure fairness in matchmaking. while Min-Heaps assisted with creating a competition list so that players are given credit for having completed their challenges first, given the order in which they completed them (thereby rewarding them). By properly testing the code, implementing build version control, and working as a team, we were able to create a stable and feature-rich multiplayer online game. These lessons can be applied to any software project where the goal is to develop a large and complex software system.